

Atividades E2E: Pedidos e Clientes

Atividades práticas para testar as telas `web/src/pages/Clientes.jsx` e `web/src/pages/Pedidos.jsx` com Playwright, seguindo o padrão já usado em `web/e2e/produtos.spec.js`.

1. Antes de começar

Como executar

Na pasta `web/`, o Playwright sobe sozinho a API (`npm run api:e2e`, porta 3000) e o front-end (`npm run dev`, porta 5173), conforme `playwright.config.js`:

```
cd web
npx playwright test
npx playwright test e2e/clientes.spec.js
npx playwright show-report
```

Padrão de isolamento

Cada teste começa com o estado inicial da API. Use o mesmo `beforeEach` do `produtos.spec.js`:

```
test.beforeEach(async ({ page, request }) => {
  const resposta = await request.post("http://localhost:3000/__reset");
  expect(resposta.status()).toBe(204);
  await page.goto("/");
});
```

Atenção: a página inicial é a de produtos. A navegação não usa rotas: o `App.jsx` tem uma barra com botões "Produtos", "Clientes" e "Pedidos". Para testar essas telas, clique no botão correspondente com `page.getByRole("button", { name: "Clientes" })` (ou "Pedidos"). O botão da aba atual fica desabilitado.

Dados iniciais (seed da API)

Entidade	Registros iniciais
Clientes	Ana Souza (ana@email.com), Bruno Lima (bruno@email.com)
Produtos	Coxinha (R\$ 5,00), <i>Pastel</i> (R\$ 8,00), Empada (R\$ 6,00)
Pedidos	#1, cliente Ana Souza, 2x Coxinha, status <code>pendente</code> , total R\$ 10,00

Mensagens de erro vindas da API

Os textos abaixo são exibidos na tela pelo `<p className="erro">`. Use-os nas asserções:

- Cliente: Nome e email são obrigatórios, Email já cadastrado, Cliente não encontrado
- Pedido: Cliente é obrigatório, Pedido deve ter ao menos um item, Itens devem ter nome, preço e quantidade válidos, Status inválido, Pedido cancelado não pode ser alterado

Documentação Oficial do Playwright

<https://playwright.dev/docs/writing-tests>

2. Atividades: Clientes.jsx

Arquivo sugerido: `web/e2e/clientes.spec.js`

Atividade C1: Listar os clientes iniciais

- **Objetivo:** verificar que a tabela mostra os clientes do seed.
- **Passos:** abrir a tela de Clientes.
- **Esperado:** heading "Clientes" visível; 3 linhas na tabela (cabeçalho + 2 clientes); células "Ana Souza" e "Bruno Lima" visíveis.
- **Conceitos:** `getByRole("heading")`, `getByRole("row")`, `getByRole("cell")`.

Atividade C2: Cadastrar um cliente novo

- **Passos:** preencher "Nome" com Carla Dias e "Email" com `carla@email.com`; clicar em "Cadastrar".

- **Esperado:** a linha com "Carla Dias" aparece na tabela com o email correto; os campos voltam a ficar vazios.

Atividade C3: Validar campos obrigatórios

- **Passos:** clicar em "Cadastrar" sem preencher nada.
- **Esperado:** mensagem Nome e email sao obrigatorios visível; nenhuma linha nova na tabela (continuam 2 clientes).
- **Discussão:** a validação está só na API. O teste cobre o comportamento visível para o usuário, não o detalhe de implementação.

Atividade C4: Impedir email duplicado

- **Passos:** cadastrar Teste com email ana@email.com .
- **Esperado:** mensagem Email ja cadastrado ; a tabela continua com 2 clientes.

Atividade C5: Editar um cliente

- **Passos:** clicar em "Editar" na linha de Bruno Lima; verificar que os campos foram preenchidos; trocar o nome para Bruno Lima Silva ; clicar em "Salvar".
- **Esperado:** o botão do formulário muda de "Cadastrar" para "Salvar" durante a edição; após salvar, a linha mostra o novo nome; o botão "Cancelar" some e o formulário é limpo.
- **Conceitos:** estado de edição, `toHaveValue` , `toBeVisible` / `not.toBeVisible` .

Atividade C6: Cancelar edição

- **Passos:** clicar em "Editar" em Ana Souza; alterar o nome no formulário; clicar em "Cancelar".
- **Esperado:** formulário limpo; o nome na tabela continua "Ana Souza" (nada foi salvo).

Atividade C7: Editar para um email já usado

- **Passos:** editar Bruno Lima e trocar o email para ana@email.com ; clicar em "Salvar".
- **Esperado:** mensagem Email ja cadastrado ; a linha de Bruno continua com o email original.

Atividade C8: Remover um cliente

- **Passos:** clicar em "Remover" na linha de Bruno Lima.
- **Esperado:** a linha desaparece (`toHaveLength(0)`); restam 1 cliente na tabela.

- **Dica:** use `page.getByRole("row", { name: /Bruno/ })` , como no teste de remoção de produtos.

Atividade C9 (desafio): Fluxo completo

Encadeie em um único teste: cadastrar "Diego", editar o nome para "Diego Matos", tentar cadastrar um segundo cliente com o mesmo email, remover "Diego Matos" e verificar a contagem final de linhas.

3. Atividades: Pedidos.jsx

Arquivo sugerido: `web/e2e/pedidos.spec.js`

Para os testes de pedido, a tela depende de três fontes de dados (clientes, produtos e pedidos). Antes de montar o pedido, confirme que o select de clientes e de produtos já foram carregados (por exemplo, com `expect(select).toContainText("Ana Souza")`).

Atividade P1: Listar os pedidos iniciais

- **Esperado:** heading "Pedidos"; uma linha de pedido com `#1` , "Ana Souza", "2x Coxinha", "R\$ 10,00" e status "pendente".
- **Conceitos:** `getByRole("cell", { name: "R$ 10,00" })` , `getByLabel("Status do pedido 1")` .

Atividade P2: Montar um pedido com um item

- **Passos:** escolher o cliente "Bruno Lima" em "Cliente"; escolher "Pastel" em "Produto"; manter quantidade 1; clicar em "Adicionar item"; clicar em "Criar pedido".
- **Esperado:** a lista de itens aparece com "1x Pastel" antes de criar; depois de criar, aparece uma nova linha com "Bruno Lima", "1x Pastel", "R\$ 8,00" e status "pendente"; o cliente e a lista de itens são limpos.

Atividade P3: Montar um pedido com vários itens e quantidades

- **Passos:** cliente "Ana Souza"; adicionar "3" x "Coxinha" (alterar o campo "Quantidade" antes de clicar); adicionar "1" x "Empada".

- **Esperado:** total de R\$ 21,00 (3 x 5 + 1 x 6); a linha do pedido mostra 3x Coxinha, 1x Empada .
- **Discussão:** o total é calculado pela API. Compare o valor exibido com a conta feita à mão.

Atividade P4: Quantidade volta a 1 após adicionar item

- **Passos:** alterar "Quantidade" para 5, adicionar um item e verificar o valor do campo.
- **Esperado:** o campo volta para 1 (comportamento de `setQuantidade(1)` em `adicionarItem`).

Atividade P5: Não criar pedido sem cliente

- **Passos:** adicionar um item sem selecionar cliente; clicar em "Criar pedido".
- **Esperado:** mensagem Cliente é obrigatório ; nenhuma linha nova na tabela.

Atividade P6: Não criar pedido sem itens

- **Passos:** selecionar cliente e clicar em "Criar pedido" sem adicionar itens.
- **Esperado:** mensagem Pedido deve ter ao menos um item .

Atividade P7: Alterar o status de um pedido

- **Passos:** no pedido #1, alterar o select "Status do pedido 1" para "pago".
- **Esperado:** o select mostra "pago" após a recarga da lista.
- **Conceitos:** `selectOption("pago")` , `toHaveValue("pago")` .

Atividade P8: Pedido cancelado não pode ser alterado

- **Passos:** mudar o pedido #1 para "cancelado"; em seguida tentar mudar para "pago".
- **Esperado:** mensagem Pedido cancelado não pode ser alterado ; o select continua em "cancelado".
- **Discussão:** o select é controlado pelo estado da lista. Verifique se, após o erro, o valor exibido é o do servidor.

Atividade P9: Remover um pedido

- **Passos:** clicar em "Remover" na linha do pedido #1.
- **Esperado:** a linha some; a tabela fica só com o cabeçalho (`toHaveLength(1)` para `row`).

Atividade P10 (desafio): Ciclo completo do pedido

Criar um pedido para "Bruno Lima" com 2x "Empada", marcar como "pago", depois tentar cancelar, cancelar, tentar voltar para "pendente" (deve dar erro) e remover. Verifique o estado da tela em cada etapa.